HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Date: February 3rd, 2025

Quiz # 1

Time Allowed: 45 minutes

Max Points: 25

Name: _____

QUESTION 1: (20 marks)

You are given a stack of unique numbers, i.e. no number appears twice in the stack. The stack is called ***StackNum*** containing 'n' elements, where 'n' is at least 2. You are also provided with an integer variable called ***in***, which is also at least 2. Assume both 'n' and 'in' are **correctly** given to you, so you may use them directly in your algorithm, without having to do any error checking or taking input. However, do declare them in the algorithm to define what they are, including the stacks and other ADT that you may use for your program

The task is to remove elements from ***StackNum*** from position, ***in*** and all multiples of ***in***, and add them in a sorted manner (ascending order) to a new stack called ***StackIns***. For example,

- Assume, ***StackNum*** = [9, 2, 5, 11, 17, 4, 1, 8, 7, 12, 19, 13, 20] and ***in*** = 3
- This means all the elements at positions – 3, 6, 9, 12, 15, … etc. should be removed from the stack, if they exist
- In terms of array indexing, these positions will refer to the indices: 2, 5, 8, 11, 14, … etc. respectively.
- In this example then, the elements to be removed will be: 5, 4, 7, 13
- These elements need to be stored in the new stack called ***StackIns*** sorted in an ascending order, thus resulting in: ***StackIns*** = [4, 5, 7, 13]
- **PLEASE NOTE the output stack,** *StackIns* **is exactly of the same size as expected from the given example, i.e. there are no extra None values in it. Hence make sure you define your stack sizes accordingly and correctly.**

You must implement the above task by writing an algorithm.

Restrictions:

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized stack of numbers with any valid index value.
- Assume the number of elements in original stack is 'n', where n > 1, so there will always be at least two numbers present in the stack
- Assume index 'in' is > 1, so there will never be a case where you will be removing all elements from the ***StackNum***, which also means the size of ***StackIns*** will always be smaller than ***StackNum***.
- ***StackIns*** must NEVER have any additional slots apart from the elements required to be in it, so in case of above example, ***StackIns*** will never be something like [4, 5, 7, 13, None] or [4, 5, 7, 13, None, None] … and etc.
- Assume None represents empty slots in all the data structures used for this question
- You are allowed to use **ONLY ONE** additional **ADT**, which cannot be the ListADT
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER: Stacks and Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structures to access the elements**

<u>QUESTION 2:</u> (5 marks)

We have the following array stored in memory: ***arr1*** = [1, 2, 3, 4, 5, 6, 19, 20, 21, 22, 11, 12, 7, 9, 11]. Assume each number is 32 bits in size, and one location of memory is 8-bits wide. Calculate the address of the element, arr1[6]

SHOW ALL THE NECESSARY STEPS/FORMULA TO ANSWER THE ABOVE QUESTION, OTHERWISE, NO MARKS WILL BE AWARDED EVEN FOR CORRECT ANSWERS!